Project Specifications: Library Database

Project Overview:

This project involves designing and building a real-world database application for a library system. The system will store and manage information about library materials, users, borrowing and returning activities, events, and personnel. The database will ensure efficient organization, retrieval, and management of library operations.

Step 2 - Entities and Attributes:

2.1 - Database design:

a. Library Items

- **Item ID** (Primary Key)
- **Title**
- **Author/Creator** (if applicable)
- **Item Type** (Book, Magazine, Journal, CD, Record, etc.)
- **Publication Year**
- **ISBN/ISSN** (if applicable)
- **Publisher**
- **Availability Status** (Available, Borrowed, Reserved)
- **Location (Shelf/Section)**
- **Digital Access Link** (if online)

b. Users

- **User ID** (Primary Key)
- **Full Name**
- **Email Address**
- **Phone Number**
- **Address**
- **Membership Type** (Regular, Student, Senior, etc.)
- **Library Card Number**
- **Account Status** (Active, Suspended)

c. Borrowing Transactions

- **Transaction ID** (Primary Key)
- **User ID** (Foreign Key)
- **Item ID** (Foreign Key)
- **Borrow Date**
- **Due Date**
- **Return Date**
- **Overdue Status** (Yes/No)
- **Fine Amount (if applicable)**

d. Fines

- **Fine ID** (Primary Key)
- **User ID** (Foreign Key)
- **Transaction ID** (Foreign Key)
- **Amount Due**
- **Paid Status** (Yes/No)
- **Payment Date**

e. Events

- **Event ID** (Primary Key)
- **Event Name**
- **Event Date & Time**
- **Event Description**
- **Target Audience** (Children, Adults, Seniors, Open to All)
- **Location (Library Room)**

f. Event Attendance

- **Attendance ID** (Primary Key)
- **Event ID** (Foreign Key)
- **User ID** (Foreign Key)

g. Personnel

- **Staff ID** (Primary Key)
- **Full Name**
- **Position** (Librarian, Assistant, IT Support, etc.)
- **Email Address**
- **Phone Number**
- **Work Schedule**

h. Future Acquisitions

- **Acquisition ID** (Primary Key)
- **Title**
- **Author/Creator**
- **Requested By (User ID or Staff ID)**
- **Approval Status** (Pending, Approved, Denied)

2.2 - Functional Requirements:

- Users can search, borrow, and return library items.
- Users can register for events.
- The system tracks fines and payments.
- Staff can manage item records and personnel details.
- The system keeps records of items that may be added in the future.

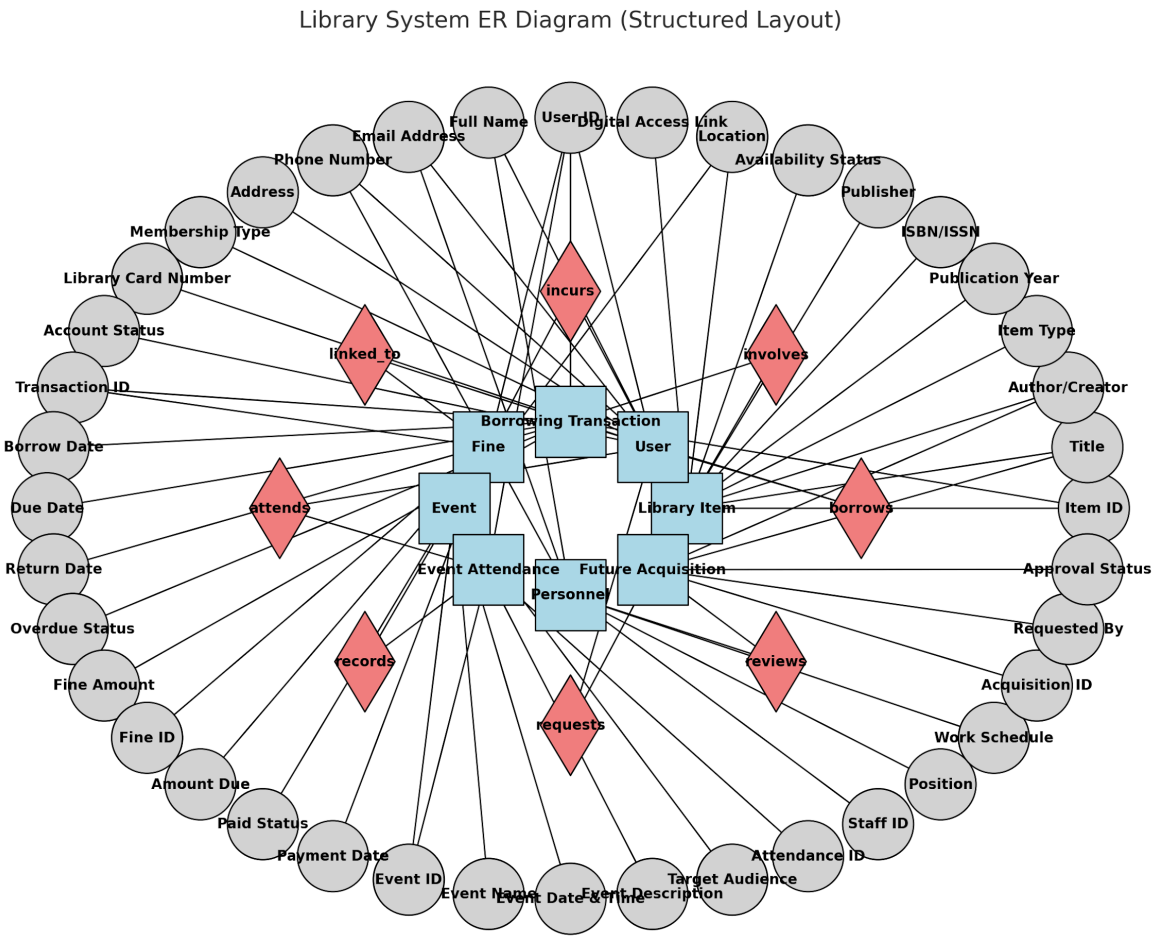
2.3 - Constraints and Assumptions:

- Users can borrow multiple items, but with a set limit.
- Items have a loan duration of 2 weeks.
- Events are free and open to registered users.
- Overdue fines are calculated based on days overdue.

2.4 - Conclusion:

This database system will organize a library's operations and ensure efficient tracking of library items, user transactions, events, and personnel records. The well-structured data model ensures effective management and accessibility, providing a functional and user-friendly experience.

Step 3 - E/R Diagram:



Step 4 - Does Your Design Allow Anomalies?:

4.1 - Understanding Functional Dependencies (FDs):

Functional dependencies (FDs) are how attributes relate within a table. A database schema should be designed to **eliminate anomalies** caused by poor FDs. The types of anomalies include:

- **Insertion Anomaly:** Inserting new data requires unnecessary dependencies.
- **Deletion Anomaly:** Deleting data causes unintended loss of other useful data.
- **Update Anomaly:** Updating one occurrence of a fact requires updating multiple rows.

4.2 - Key Functional Dependencies in the Library Database:

Below are the main functional dependencies found in our schema:

Library Items (Books, CDs, Magazines, etc.)

- `ItemID → (Title, Author, Type, Status, Location)`
- `ISBN → (Title, Author, Publisher, Year)`
- `ItemID → ISBN`

Users

- `UserID → (FullName, Email, Phone, Address, MembershipType, CardNumber, AccountStatus)`

Borrowing Transactions

- `TransactionID → (UserID, ItemID, BorrowDate, DueDate, ReturnDate, OverdueStatus, FineAmount)`
- `(UserID, ItemID) → BorrowDate`

Fines

- `FineID → (UserID, TransactionID, Amount, PaidStatus, PaymentDate)`
- `TransactionID → FineAmount`

Events & Attendance

- `EventID → (EventName, EventDate, Description, Audience, Location)`
- `(UserID, EventID) → AttendanceID`

Personnel Records

- `StaffID → (FullName, Position, Email, Phone, WorkSchedule)`

Future Acquisitions

- `AcquisitionID → (Title, Author, RequestedBy, ApprovalStatus)`

4.3 - Normalization & BCNF Analysis:

To avoid anomalies, our database schema should be **at least in BCNF (Boyce-Codd Normal Form)**, which requires:

1. **1NF (First Normal Form):** No repeating groups or multivalued attributes.
2. **2NF (Second Normal Form):** No partial dependencies
3. **3NF (Third Normal Form):** No transitive dependencies
4. **BCNF:** Every determinant is a candidate key

Analysis of Our Schema:

- **Library Items Table**: `ItemID` is the primary key, and all attributes depend **only** on `ItemID` → **Meets BCNF.**
- **Users Table**: `UserID` uniquely determines all other attributes → **Meets BCNF.**
- **Borrowing Transactions Table**: `TransactionID` is the primary key, ensuring that `UserID` and `ItemID` are linked correctly → **Meets BCNF.**
- **Fines Table**: `FineID` uniquely identifies fines, and `TransactionID → FineAmount` ensures correct relationships → **Meets BCNF.**
- **Events Table**: `EventID` is the primary key, and all other attributes are fully functionally dependent on it → **Meets BCNF.**
- **Personnel Table**: `StaffID` uniquely determines all personnel attributes → **Meets BCNF.**
- **Future Acquisitions Table**: `AcquisitionID` uniquely identifies acquisitions → **Meets BCNF.**

4.4 - Conclusion: No Anomalies Exist

Since our final design meets BCNF, there are no terrible functional dependencies, and therefore, No insertion anomalies, no deletion anomalies, and no update anomalies

Therefore, our database design does not allow anomalies, and decomposition is unnecessary.

Steps 5-7:

- The database is created and the tables are populated in ‘library.py’.
- The database application is created and managed in ‘LibraryApp.py’.
- To use our application, use your terminal and run ‘python LibraryApp.py’.
- If you want to recreate the database, remove the ‘library.db’ file, then use ‘python library.py’ to create a new database.